

VIST/ECEN 489 - Final Project Report

Golf Ball Trajectory Reconstruction using Physics-constrained Optimization

Kris Kafle - kriskafle@tamu.edu - Physics Simulation and Pipeline

Leo Luo - leoluo2016@tamu.edu - Visualization and Calculation Pipeline

Nathan Rao - nathan_rao@tamu.edu - Computational Physics and Equations Research

1 Executive Summary

This project presents a GPU-accelerated system for estimating the trajectory of a golf ball and carry distance using open source computer vision and physics-based simulation.

From the face-on view of a golfer, initial launch conditions such as ball speed and launch angle can be extrapolated using computer vision techniques. Since spin rate cannot be directly measured from a face-on view, it is approximated based on the selected golf club using common data. These parameters are then used in a physics-based trajectory simulation that accounts for aerodynamics effects such as drag, spin, magnus effect, and basic projectile motion. The spin rate is swept within a certain range around the estimated value, and it is observed how it affects the ball flight depending on the other initial parameters.

CUDA was used to accelerate the per-pixel image preprocessing stage of the tracker. Specifically, GPU kernels generated the color mask, computed frame-to-frame motion, applied the wedge mask, and performed dilation, while the higher-level blob detection and tracking logic remained on the CPU. CUDA reduced the kernel-only per-pixel processing time compared to the CPU pixel stage, showing that the thresholding, masking, and dilation operations benefit from GPU parallelism. However, the total program speedup was smaller because video I/O, display, memory transfers, and CPU-based blob detection still remained part of the overall pipeline.

CUDA was also used to accelerate the calculations for the simulation in order to sweep more spin rate values, allowing rapid evaluation of multiple trajectories. Parallelization was observed to significantly cut the time of calculation for larger numbers of calculations.

2 Introduction

The ability to accurately measure the trajectory of a golf ball and certain parameters of the ball flight is very important for performance analysis of golfers. Commercial systems use expensive radar or laser technology to accurately measure these values. However, these systems are very expensive, and are only limited to high-income individuals or professional golfers. Other systems such as mobile applications are often inaccurate, inconvenient, and still require payment. This project aims to develop a low-cost alternative using a single face-on camera combined with GPU acceleration to accurately track the ball flight statistics of a golf ball.

Our original goal of extrapolating this data from a down-the-line view (behind the golfer) proved to be almost impossible due to challenges with noise in our computer vision model, attempting to map the 3D environment, and major physics constraints. We instead decided to switch to extrapolating data from a face-on view where it is much easier to find the launch angle and ball speed frame-by-frame. With this data, we improved an existing physics model (Burglund and Street) and provided it with the data we gathered along with an estimated spin rate to create a 2D simulation of the trajectory of the golf ball. This estimated spin rate is swept across a range of values to observe the varying carry distances depending on the spin rate.

The usage of CUDA was also important to our project by utilizing parallelization to significantly speed up calculations in our model. The CUDA per-pixel processing, especially for larger videos, will provide a noticeable speedup compared to the CPU benchmark.

3 Methodology

Physics Considerations

- Displacement in Pixels: $d_{pixels} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
- Conversion Factor: $yards_{per\ pixel} = \frac{d_{ball}(inches)}{d_{ball}(pixels)} \cdot \frac{1}{36}$
- Displacement in Yards: $d_{yards} = d_{pixels} \cdot scale_{yards-per-pixel}$
- Solving for Velocity: $v_x = \frac{x_2 - x_1}{\Delta t}$, $v_y = \frac{y_2 - y_1}{\Delta t}$, $\Delta t = \frac{1}{fps}$, $v = \sqrt{v_x^2 + v_y^2}$
- Launch Angle: $\theta = \tan^{-1}\left(\frac{v_y}{v_x}\right)$
- Equation of Motion using Sum of Forces:
$$m \frac{dv}{dt} = (0, -mg) - \frac{1}{2} \rho C_d A v^2 \hat{v} + \frac{1}{2} \rho C_l A v^2 \hat{n}$$

Spin approximated from golf club, drag force approximated, ideal conditions assumed (flat surface, negligible wind), simple projectile motion due to face-on viewpoint

Visualization Pipeline

- A face-on golf swing video was processed frame-by-frame using OpenCV.
- The user selected a launch ROI around the ball/tee location in the first frame. This ROI was used to determine the initial ball position and define the launch origin.
- The selected ROI was checked against predefined HSV thresholds for white and yellow golf balls. The color range with the higher pixel count was used for the rest of the tracking process.
- For each frame, the image was converted into a binary color mask that isolated pixels matching the selected golf ball color.
- Consecutive color masks were compared using frame differencing to isolate moving pixels rather than stationary white/yellow objects.

- A wedge-shaped search region was created from the launch origin to restrict detection to the expected launch direction of the golf ball, and the frame-difference mask was combined with the wedge mask to produce a final motion_in_wedge binary image.
- Blob detection was performed on the final binary mask using OpenCV contour detection. Each blob was filtered by area, radius, and wedge position.
- The valid blobs were sorted by horizontal position, and the two rightmost blobs were selected as the candidate two-blob launch pair.
- Once the desired valid pair number was reached, the pixel displacement between the two blobs was used to calculate horizontal velocity, vertical velocity, and launch angle.
- An approximate real-world speed was calculated using the known golf ball diameter to estimate meters per pixel.

Simulation and Parameter Sweep

- Spin rate was swept from ± 1000 rpm of the spin rate of what is expected for a given club while holding the initial ball speed and launch angle constant from what was found from the computer vision extrapolation
- The number of parameters swept within that range varied from 10 to 80 different trajectories. From that, the carry distance and trajectory is found for each respective spin rate.
- The physics model used was based on an existing MATLAB simulation (Burglund and Street 17-19) which was translated into C++ code and then sped up with CUDA.

4 Results

CPU vs. CUDA Implementation

Metric	CPU	CUDA	Speedup
Frames processed	806	806	—
Total runtime	8.985 s	8.298 s	1.08x
Average dilation time/frame	0.166 ms	0.023 ms	7.22x
Pixel/kernel stage time/frame	1.234 ms	0.164 ms	7.52x
Pixel/kernel total time	0.995 s	0.132 s	7.54x
Blob detection time /frame	0.210 ms	0.246 ms	0.85x
Blob detection total time	0.169 s	0.199 s	0.85x

For a 720x1280 slow-motion recording, the CUDA implementation produced the same detection result as the CPU version while reducing the measured pixel-processing computation from 1.234 ms/frame to 0.164 ms/frame, giving about a 7.5 $\times$ speedup for the kernel-only image-processing stage. Should there be a dilation option specified, CUDA also has a significant speedup compared to CPU (7.22x), ramping up if dilation iterations are increased. However, the total runtime only improved from 8.985 s to 8.298 s, or about 1.08 $\times$ overall (without dilation). This smaller end-to-end improvement occurs because the full program still uses CPU video reading, GPU memory transfers, CPU contour/blob detection, drawing, video writing, and OpenCV display overhead. Therefore, CUDA significantly accelerates the parallel per-pixel computation, but the full application remains limited by CPU-side and I/O operations. This effectively shows the CUDA acceleration portion of the project and how it can be applied to

filters per frame to reduce total runtime. The more filters applied, the greater the speedup compared to CPU. Should the size/quality of the video increase, there should be an even greater speedup.

Spin Rate Sweep Simulation

Trajectories Computed (n)	CPU (ms)	GPU (ms)	Speedup
10	1.511	5.54	0.273x
20	2.876	5.47	0.526x
40	5.505	5.94	0.927x
80	12.218	7.11	1.718x

In the simulation, as more trajectories were computed within the parameter sweep, the CPU exhibited linear growth in runtime. Overhead dominated for the GPU when computing a smaller number of trajectories. At 80 trajectories the GPU runtime was finally faster than the CPU, giving about a 1.718x speedup. If an even larger number of trajectories were to be computed, the GPU speedup would be even larger as the CPU runtime would continue to scale linearly, while the GPU runtime would scale significantly slower. This demonstrates the benefit of parallelization with GPUs, where it is most efficient when the number of computations are significantly large. Additionally, the carry distances calculated by the simulation were very close to the real distance that was expected, and it modeled the expected behavior when spin is increased or reduced given the initial conditions.

5 Discussion

The results show that CUDA was effective for the project's main acceleration objective: improving the highly parallel per-pixel preprocessing stage used for thresholding, motion masking, wedge filtering, and dilation. The GPU implementation reduced the kernel-only processing time compared to the CPU version, confirming that these operations are well suited for CUDA parallelism. However, the overall runtime improvement was more limited because the pipeline still depends on CPU-based video I/O, display, memory transfers, and OpenCV contour/blob detection. This means the project met its goal of demonstrating GPU acceleration for the image-processing stage, but it also revealed that end-to-end performance is constrained by non-GPU portions of the tracker. Future work could focus on moving blob detection to the GPU, reducing CPU-GPU memory transfers, using GPU-based video decoding/encoding, and testing the tracker across more swing videos and lighting conditions. Other limitations include requirement of FPS metadata for ball speed, requirement of high FPS for ball to be in frame for at least 2 frames, and extraneous noise even in the restricted wedge region of interest either due to color of club, clothing, shoes, or camera movement. For our 2 test cases of yellow and white balls in a controlled and still environment, our results accurately matched real life ball speeds and carry distances, at virtually no cost compared to launch monitors.

CUDA was also effective for simulating the golf ball trajectory for multiple spin rate values. At a small number of computations, the GPU had no added benefit as the overhead dominated the runtime. As the number of simulation scales up, the speedup over CPU would continue to grow. Furthermore, the simulation could be expanded to be 3D to handle curvature such as a fade or draw. However, this would require more advanced physics models as values for another dimension would have to be calculated which may require more initial conditions.

6 Conclusion

The project showed that CUDA can significantly reduce the computation time of the per-pixel masking and dilation stages in a golf ball tracking pipeline. However, the overall speedup was limited by CPU-based tasks such as video I/O, display, and blob detection. These results are relevant because they show where GPU acceleration is most useful and where the pipeline still has bottlenecks. Practically, this approach could support faster sports video analysis and future real-time tracking systems if more of the pipeline is moved onto the GPU. Simulation wise, CUDA showed speedup when larger numbers of computations were done while there was no benefit for small numbers of computations due to overhead.

7 Future Work

Future work could extend this project by testing higher-resolution videos, such as slow-motion 4K footage, where the larger number of pixels may allow CUDA to show greater speedup in the masking and dilation stages. Further testing could determine whether a more fully GPU-based pipeline would be practical for real-time golf ball tracking and launch-condition estimation. The exploration of building OpenCV from source using Cmake using “WITH_CUDA=ON” could make it easier to implement GPU accelerated functions like linear, non-linear, and morphological filters. Currently, we assume the camera is stationary, which allows us to use frame differencing with a color filter to track the moving golf ball. Potential future work would be to use a different approach since any camera movement will induce many false positives, even when restricting our region of interest with manual selection of the tracked ball.

8 References

“CUDA.” *OpenCV*, opencv.org/platforms/cuda/.

OpenCV. “OpenCV: OpenCV Modules.” Docs.opencv.org, docs.opencv.org/4.x/index.html.

“Design Notes - Golf Physics P3.” *Wwww.tutelman.com*,
www.tutelman.com/golf/design/swing3.php.

“Physics.” Golf-Simulators.com, 2026,
www.golf-simulators.com/physics.htm?srsId=AfmBOop_LgmUfQym8E6DtOW6lKuo_mqw2vJEQuQzfJaQ32SJhs5CazCWF. Accessed 1 May 2026.

Burglund, Brett, and Ryan Street. “Mathematics Department.” *Union College*, 13 May 2011,
www.math.union.edu/~wangj/courses/previous/math238w13/Golf%20Ball%20Flight%20Dynamics2.pdf.

9 Appendix

CPU compile and run commands in WSL:

```
g++ face_on_timed.cpp -o ball_tracker $(pkg-config --cflags --libs opencv4)
./ball_tracker
```

CUDA compile and run commands in WSL:

```
nvcc --expt-relaxed-constexpr -std=c++17 two_blob_tracking_cuda_timed.cu -o
ball_tracker_cuda $(pkg-config --cflags --libs opencv4)
./ball_tracker_cuda
```



Figure 1. White color filter, frame differencing, wedge restriction, and final capture pipeline at impact (respectively)

9.1 Latest Ball Flight m-file

```
function Golfball
v0_mph = 128; %109, 128, 180
launch_deg = 19.2; %25.3, 19.2, 15.6
spin_rpm = 5400; %8167, 5400, 3000
opts = odeset('Events', @hitGround, 'RelTol', 1e-8, 'AbsTol',
1e-10);
v0 = v0_mph*5280/3600;
theta = deg2rad(launch_deg);
vx0 = v0*cos(theta);
vy0 = v0*sin(theta);
[t,x] = ode45(@(t,x) flightg(t,x,spin_rpm), [0,20],
[0,vx0,0.01,vy0], opts);
plot(x(:,1),x(:,3),'-ob');
title('Golf Ball Model')
xlabel('Distance X (feet)')
ylabel('Height Y (feet)')
grid on
axis equal
carry_ft = x(end,1);
carry_yd = carry_ft/3;
fprintf('Carry distance = %.2f ft = %.2f yd\n', carry_ft,
carry_yd);
```

```

fprintf('Flight time = %.2f s\n', t(end));
end
function xprime = flightg(t,x,spin_rpm)
g = 32.174;
rho = 0.0023769;
r = 1.68/24;
A = pi*r^2;
m = 0.1012/32.174;
v = sqrt(x(2)^2 + x(4)^2);
v_safe = max(v,1e-6);
v_mph = v*3600/5280;
Cd_high = 0.075;
Cd_low = 0.34;
v_transition = 130;
v_width = 15;
Cd = Cd_high + (Cd_low - Cd_high)/(1 + exp((v_mph -
v_transition)/v_width));
omega0 = spin_rpm*2*pi/60;
spin_tau = 5 + 7/(1 + exp(-(v_mph - 130)/15));
omega = omega0*exp(-t/spin_tau);
spinRatio = omega*r/v_safe;
Cl_slope = 1.4;
Cl_max = 0.5;
Cl = Cl_slope*spinRatio;
Cl = min(max(Cl,0),Cl_max);
if x(4) < 0
    descentFactor = 1 - 0.45/(1 + exp((v_mph - 140)/10));
    Cl = Cl*descentFactor;
end
Fd = 0.5*rho*A*Cd*v^2;
Fl = 0.5*rho*A*Cl*v^2;
drag_x = -Fd*x(2)/v_safe;
drag_y = -Fd*x(4)/v_safe;
lift_x = -Fl*x(4)/v_safe;
lift_y = Fl*x(2)/v_safe;
xprime = zeros(4,1);
xprime(1) = x(2);
xprime(2) = (drag_x + lift_x)/m;
xprime(3) = x(4);
xprime(4) = -g + (drag_y + lift_y)/m;
end

```

```
function [value, isterminal, direction] = hitGround(t,x)
value = x(3);
isterminal = 1;
direction = -1;
end
```

10 Signature

Kris Kafle - kriskafle@tamu.edu - Physics Simulation and Pipeline

Leo Luo - leoluo2016@tamu.edu - Visualization and Calculation Pipeline

Nathan Rao - nathan_rao@tamu.edu - Computational Physics and Equations Research